The Globe oposiciones

Bloque 3 - Tema 6 ejercicios 3

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS

Soluciones

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

Enunciado 1

Definir y probar las siguientes clases:

- **Direccion**:
  - Atributos: calle, ciudad, código postal, país.
  - Constructores predeterminado y parametrizado.

- **Persona**:
  - Atributos: nombre, apellidos, NIF y dirección.
  - Métodos getters y setters.
  - Esta clase implementa la interface Humano, con un método identificate(), que muestra el tipo de la clase que lo implementa (el tipo de persona, en este caso).

- **Estudiante**: subclase de Persona.
  - Atributos: ID de estudiante.
  - Constructores: predeterminado y constructor parametrizado que admita el ID.
  - Métodos getters y setters y toString().

- **Profesor: subclase de Persona.**
  - Atributos: despacho.
  - Constructores: predeterminado y constructor parametrizado que admita el despacho.
  - Métodos getters y setters y toString()

Cree una lista de personas (con la clase Vector) y, posteriormente, añada varios alumnos y varios profesores a la lista y pruebe las operaciones definidas.

SOLUCIÓN

```
public class Direccion {
    private String calle;
    private String ciudad;
    private String pais;

    public Direccion() { }

    public Direccion(String calle, String ciudad, String pais) {
        this.calle = calle;
        this.ciudad = ciudad;
        this.pais = pais;
    }

    public String getCalle() {
        return calle;
    }
    public void setCalle(String calle) {
```

```
        this.calle = calle;
    }

    public String getCiudad() {
        return ciudad;
    }

    public void setCiudad(String ciudad) {
        this.ciudad = ciudad;
    }

    public String getPais() {
        return pais;
    }

    public void setPais(String pais) {
        this.pais = pais;
    }

    @Override
    public String toString() {
        return "Direccion{" + "calle=" + calle + ", ciudad=" + ciudad +
", pais=" + pais + '}';
    }
}

public interface Humano {
    public int identificate();
}

public class Persona implements Humano {
    private String nombre;
    private String apellidos;
    private String nif;
    private Direccion direccion;

    public Persona() {}

    public Persona(String nombre, String apellidos, String nif, Direccion
direccion) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.nif = nif;
        this.direccion = direccion;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getApellidos() {
        return apellidos;
    }

    public void setApellidos(String apellidos) {
```

```
        this.apellidos = apellidos;
    }

    public String getNif() {
        return nif;
    }

    public void setNif(String nif) {
        this.nif = nif;
    }

    public Direccion getDireccion() {
        return direccion;
    }

    public void setDireccion(Direccion direccion) {
        this.direccion = direccion;
    }

    public int identificate() {
        if(this instanceof Estudiante)
            return 0; //estudiante
        else if(this instanceof Profesor)
                return 1; //profesor
             else
                return 2; //persona
    }

    public String toString() {
        return "Persona{" + "nombre=" + nombre + ", apellidos=" +
apellidos + ", nif=" + nif + '}';
    }
}

public class Estudiante extends Persona {
    private int id;

    public Estudiante() {}

    public Estudiante(int id, String nombre, String apellidos, String
nif, Direccion direccion) {
        super(nombre, apellidos, nif, direccion);
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
    @Override
    public String toString() {
        return "Estudiante{" + "id=" + id + super.toString() + "}";
    }
}
```

```
public class Profesor extends Persona {
    private int despacho;

    public Profesor() {}

    public Profesor(int despacho, String nombre, String apellidos, String
nif, Direccion direccion) {
        super(nombre, apellidos, nif, direccion);
        this.despacho = despacho;
    }

    public int getDespacho() {
        return despacho;
    }

    public void setDespacho(int despacho) {
        this.despacho = despacho;
    }

    @Override
    public String toString() {
        return "Profesor{" + "despacho=" + despacho + super.toString() +
"}";
    }
}


import java.util.Vector;

public class PruebaEj3 {
    public static void main(String[] args) {
        Vector<Persona> personas = new Vector<Persona>();

        personas.add(new
Estudiante(1234,"Nombre1","Apellidos1","1234567A",
            new Direccion("calle1","Granada","España")));
        personas.add(new
Estudiante(2345,"Nombre2","Apellidos2","2345678B",
            new Direccion("calle2","Madrid","España")));
        personas.add(new
Estudiante(3456,"Nombre3","Apellidos3","3456789C",
            new Direccion("calle3","Cordoba","España")));
        personas.add(new Profesor(101,"Nombre4","Apellidos4","7654321D",
            new Direccion("calle4","Granada","España")));
        personas.add(new Profesor(202,"Nombre5","Apellidos5","8765432E",
            new Direccion("calle5","Sevilla","España")));

        for(Persona aux : personas) {
            System.out.println(aux);
            System.out.println(aux.identificate());
        }
    }
}
```

Enunciado 2

Defina una clase NIF que se usará para mantener DNIs con su correspondiente letra. Los atributos serán el número de DNI (entero largo) y la letra que le corresponde. La clase dispondrá de los siguientes métodos:

- Constructor predeterminado que inicialice el no de DNI a 0 y la letra a espacio en blanco (será un NIF no válido).

- Constructor que reciba el DNI y establezca la letra que le corresponde.

- getters y setters para el número de DNI (que ajuste automáticamente la letra).

- leer(): que pida el número de DNI (ajustando automáticamente la letra).

- Método que nos permita mostrar el NIF (ocho dígitos, un guion y la letra en mayúsculas; por ejemplo: 00395469-F)

La letra se calculará con un método auxiliar (privado) de la siguiente forma: se obtiene el resto de la división entera del número de DNI entre 23 y se usa la siguiente tabla para obtener la letra que corresponde:

0-T	1-R	2-W	3-A	4-G	5-M	6–Y	7-F	8-P	9-D	10-X	11-B	12-N
13–J	14-Z	15-S	16-Q	17-V	18-H	19-L	20–C	21-K	22–E			

SOLUCIÓN

```
public class Nif {
    private long dni;
    private String letra;
    private String [] letras =
{"T","R","W","A","G","M","Y","F","P","D","X",
            "B","N","J","Z","S","Q","V","H","L","C","K","E"};

    public Nif() {
        dni = 0;
        letra = " ";
    }

    public Nif(long dni, String letra) {
        this.dni = dni;
        this.letra = letra;
    }

    public long getDni() {
        return dni;
    }

    public void setDni(long dni) {
        this.dni = dni;
    }
```

```
    public String getLetra() {
        return letra;
    }

    public void setLetra(String letra) {
        this.letra = letra;
    }

    public void leer(long dni) {
        this.dni = dni;
        calcularLetra();
    }

    public String mostrar() {
        return dni + "-" + letra;
    }

    private void calcularLetra() {
        letra = letras[(int)dni%23];
    }
}
```

Enunciado 3

En un puerto se alquilan amarres para barcos de distinto tipo. Para cada alquiler se guarda el nombre y DNI del cliente, las fechas de inicio y de fin de alquiler, la posición del amarre y el barco que lo ocupará. Un barco se caracteriza por su matrícula, su eslora en metros y año de fabricación.

Un alquiler se calcula multiplicando el número de días de ocupación (incluyendo los días inicial y final) por un módulos función de cada barco (obtenido multiplicando por 10 los metros de eslora) y por un valor fijo (12€).

Se pretende diferenciar la información de algunos tipos de barcos:

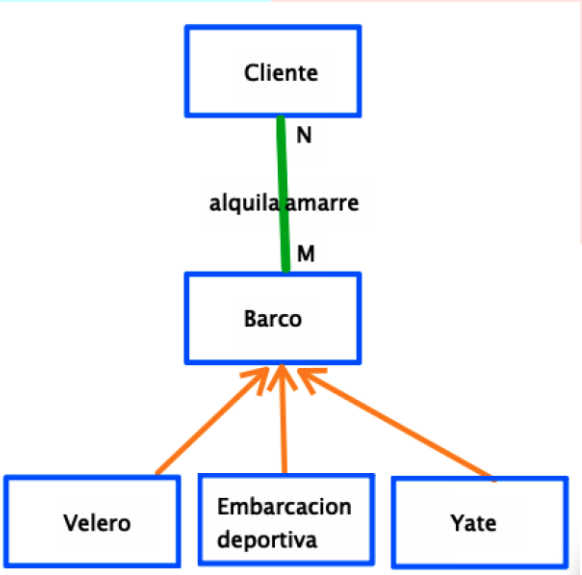
- número de mástiles para veleros.
- potencia en CV para embarcaciones deportivas a motor.
- potencia en CV y número de camarotes para yates de lujo.

El módulo de los barcos de un tipo especial se obtiene como el módulo normal mas:

- el número de mástiles para veleros.
- la potencia en CV para embarcaciones deportivas a motor.
- la potencia en CV más el número de camarotes para yates de lujo.

Elabore el diagrama entidad-relación según lo descrito y desarrolle en Java las clases implementando los métodos que permitan calcular el alquiler de cualquier tipo de barco.

SOLUCIÓN



```
public class Barco {
    private String matricula;
    private double eslora;
    private int fab;

    public Barco() {}

    public Barco(String matricula, double eslora, int fab) {
        this.matricula = matricula;
        this.eslora = eslora;
        this.fab = fab;
    }

    public String getMatricula() {
        return matricula;
    }

    public void setMatricula(String matricula) {
        this.matricula = matricula;
    }

    public double getEslora() {
        return eslora;
    }

    public void setEslora(double eslora) {
        this.eslora = eslora;
    }

    public int getFab() {
        return fab;
    }

    public void setFab(int fab) {
        this.fab = fab;
    }

    public double modulo() {
        return eslora * 10 * 12;
    }
}

public class Velero extends Barco {
    private int mastiles;

    public Velero() {}

    public Velero(int mastiles, String matricula, double eslora, int fab)
{
        super(matricula, eslora, fab);
        this.mastiles = mastiles;
    }

    public int getMastiles() {
        return mastiles;
    }
```

```
    public void setMastiles(int mastiles) {
        this.mastiles = mastiles;
    }

    public double modulo() {
        return super.modulo() + mastiles;
    }
}

public class EmbarcacionDeportiva extends Barco {
    private double potencia;

    public EmbarcacionDeportiva() {}

    public EmbarcacionDeportiva(double potencia, String matricula, double
eslora, int fab) {
        super(matricula, eslora, fab);
        this.potencia = potencia;
    }

    public double getPotencia() {
        return potencia;
    }

    public void setPotencia(double potencia) {
        this.potencia = potencia;
    }

    public double modulo() {
        return super.modulo() + potencia;
    }
}

public class Yate extends Barco {
    private double potencia;
    private int camarotes;

    public Yate() {}

    public Yate(double potencia, int camarotes, String matricula, double
eslora, int fab) {
        super(matricula, eslora, fab);
        this.potencia = potencia;
        this.camarotes = camarotes;
    }

    public double getPotencia() {
        return potencia;
    }

    public void setPotencia(double potencia) {
        this.potencia = potencia;
    }
    public int getCamarotes() {
        return camarotes;
    }
```

```
    public void setCamarotes(int camarotes) {
        this.camarotes = camarotes;
    }

    public double modulo() {
        return super.modulo() + potencia + camarotes;
    }
}

public class Cliente {
    private String nombre;
    private long dni;

    public Cliente() {}

    public Cliente(String nombre, long dni) {
        this.nombre = nombre;
        this.dni = dni;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public long getDni() {
        return dni;
    }

    public void setDni(long dni) {
        this.dni = dni;
    }
}


import java.util.Date;

public class Alquiler {
    private Cliente cliente;
    private Date fechaInicio;
    private Date fechaFin;
    private double posicion; //amarre
    private Barco barco;
    private double precio;

    public Alquiler() {}

    public Alquiler(Cliente cliente, Date fechaInicio, Date fechaFin,
double posicion, Barco barco) {
        this.cliente = cliente;
        this.fechaInicio = fechaInicio;
        this.fechaFin = fechaFin;
        this.posicion = posicion;
        this.barco = barco;
    }
```

```
    public Cliente getCliente() {
        return cliente;
    }

    public void setCliente(Cliente cliente) {
        this.cliente = cliente;
    }

    public Date getFechaInicio() {
        return fechaInicio;
    }

    public void setFechaInicio(Date fechaInicio) {
        this.fechaInicio = fechaInicio;
    }

    public Date getFechaFin() {
        return fechaFin;
    }

    public void setFechaFin(Date fechaFin) {
        this.fechaFin = fechaFin;
    }

    public double getPosicion() {
        return posicion;
    }

    public void setPosicion(double posicion) {
        this.posicion = posicion;
    }

    public Barco getBarco() {
        return barco;
    }

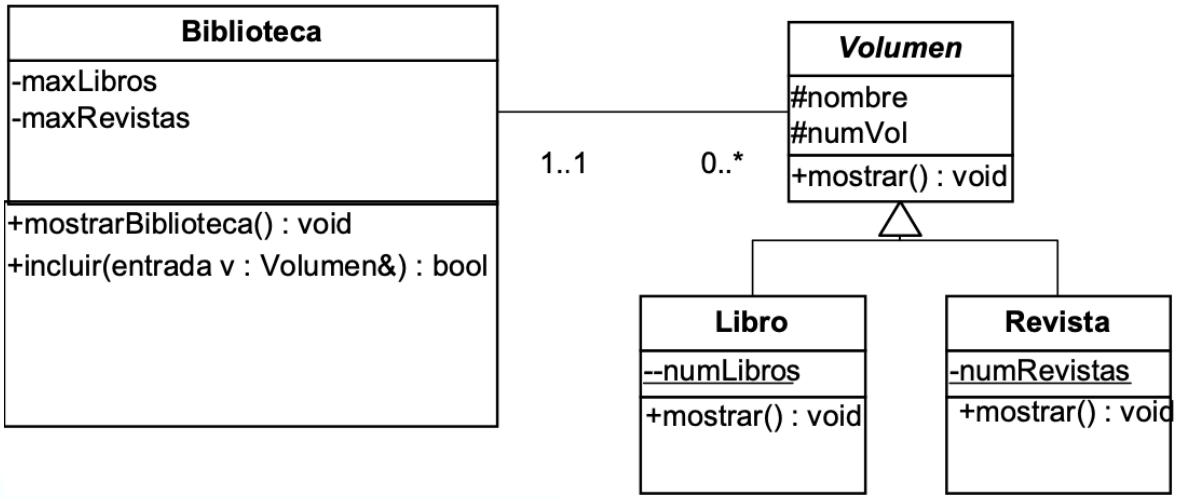
    public void setBarco(Barco barco) {
        this.barco = barco;
    }

    public double getPrecio() {
        return precio;
    }

    public void precioAlquiler() {
        precio = (fechaFin.getTime() - fechaInicio.getTime() + 1) /
   86400000 * barco.modulo();
    }
}
```

Enunciado 4

A partir del siguiente diagrama de clases, que modela un Sistema de gestión para una Biblioteca, defina las clases en Java mostradas:



Defina la función mostrar() de la clase Volumen, así como la de la clase Revista teniendo en cuenta que dicha función debe comportarse de manera polimórfica.

Implemente una función incluir(...) en la clase Biblioteca que permita añadir con el mismo código tanto una revista como un libro que se le pase como parámetro.

SOLUCIÓN

```
import java.util.ArrayList;

public class Biblioteca {
    private int maxLibros;
    private int maxRevistas;
    private int contLibros;
    private int contRevistas;

    private ArrayList<Volumen> volumenes;

    public Biblioteca() {}

    public Biblioteca(int maxLibros, int maxRevistas) {
        this.maxLibros = maxLibros;
        this.maxRevistas = maxRevistas;
    }

    public void mostrarBiblioteca() {
        for (Volumen v : volumenes)
            v.mostrar();
    }
```

```
    public boolean incluir(Volumen v) {
        if(v instanceof Libro && contLibros < maxLibros) {
            contLibros++;
            return volumenes.add(v);
        }

        if(v instanceof Revista && contRevistas < maxRevistas) {
            contRevistas++;
            return volumenes.add(v);
        }
        return false;
    }
}

public abstract class Volumen {
    protected String nombre;
    protected int numVol;

    public abstract void mostrar();
}

public class Libro extends Volumen {
    private int numLibros;

    public void mostrar() {
        System.out.println("Mostrando libro");
    }
}

public class Revista extends Volumen {
    public int numRevistas;

    public void mostrar() {
        System.out.println("Mostrando revista");
    }
}
```

Enunciado 5

Responda a las siguientes cuestiones:

a) Dado el siguiente código:

```
Dispositivo disp = new Impresora();
disp.getNombre();
```

`Impresora` es una subclase de `Dispositivo`. ¿Cuál de estas dos clases debe tener la definición del método `getNombre` en su código para que compile?

Tenemos tres opciones:

- El método getNombre está definido en las dos clases: compila.
- El método está definido solo en la superclase (Dispositivo): compila.
- El método está definido solo en la clase hija (Impresora): no compila

Si ambas clases tienen una implementación de método `getNombre`, ¿cuál de ellos ejecutará?

Si ambas lo tienen, se ejecutará el método getNombre de la clase Impresora.

b) Si tenemos una clase `Estudiante` que no tiene una superclase declarada, no sobrescribe el método `toString` y considerando el siguiente código:

```
Estudiante est = new Estudiante();
String s = est.toString();
```

¿Compilarán estas líneas de código?
Sí.

¿Qué sucederá exactamente cuando intente ejecutar estas líneas de código?
Se llamará al método toString de la clase Object.

Suponiendo ahora que la clase `Estudiante` sobrescribe el método `toString` de tal modo que devuelve el nombre del estudiante. Explique detalladamente la ejecución del siguiente código (si es que compila correctamente):

```
for(Object est : miLista)
   System.out.println(est);
```

Para cada estudiante de miLista se muestra por pantalla el nombre del estudiante.

c) ¿Puede una clase abstracta tener métodos concretos (no abstractos)?
Sí, no existe restricción para que no se pueda. De hecho, una clase abstracta puede definir todos sus métodos como concretos.

¿Puede una clase concreta tener métodos abstractos?
No. Una clase concreta es aquella que puede instanciarse. Si tiene un método abstracto obliga a que la clase sea abstracta, por lo que no es posible tal situación

¿Se puede tener una clase abstracta sin métodos abstractos? Justifique sus respuestas.
Sí. Tal y como se ha indicado en la primera respuesta puede ocurrir que todos los métodos de una clase abstracta sean concretos sin violar en ningún momento la abstracción de la misma.

d) Observe el siguiente código. Se tienen 5 tipos (clases o interfaces): U, G, B, Z y X y una variable de cada uno de estos tipos:

```
U u; G g; B b; Z z; X x;
```

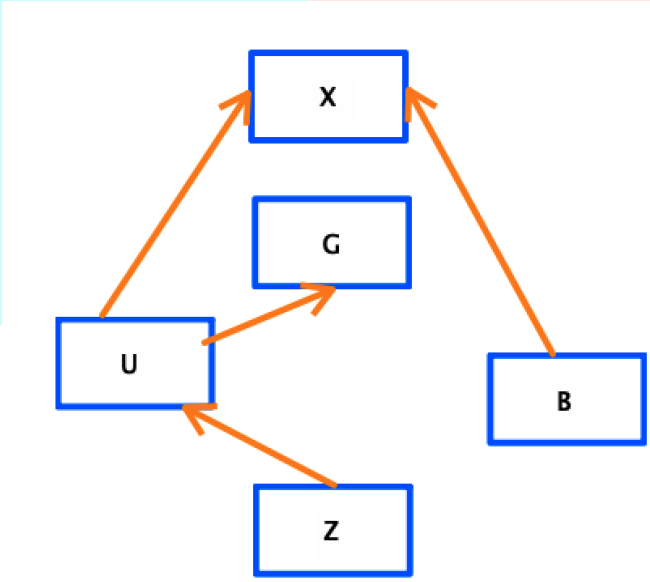
Las siguientes sentencias son todas legales (asumimos que todas compilan):

```
u = z;
x = b;
g = u;
x = u;
```

Las siguientes sentencias son todas ilegales (se producen errores de compilación):

```
u = b;
x = g;
b = u;
z = u;
g = x;
```

¿Qué puede decir sobre los tipos y sus relaciones?
Z es una subclase de U.
B es una subclase de X.
U es una subclase de G.
U es una subclase de X.



¿Qué relaciones existen entre ellas?
Relaciones de herencia.

e) Queremos intercambiar el valor de dos variables enteras a y b. Para ello escribimos el siguiente método:

```
public void intercambiar (int i1, int i2)
{
   int temp = i1;
   i1 = i2;
   i2= temp;
}
```

Si invocamos este método sobre nuestras variables a y b:

```
intercambiar (a, b);
```

¿Se intercambian realmente los valores de a y b después de la invocación? Justifique la respuesta.
No, porque los argumentos se han pasado por valor y no por referencia, por lo que los cambios no se reflejarán fuera del método.